

Blockwise Adaptive Looped Transformers

Het

Department of Computer Science
Binghamton University
Binghamton, New York, USA

ABSTRACT

A standard transformer applies the same sized stack of layers to every input, essentially spending the same compute on easy and hard problems. Looped, or weight-tied, transformers, instead reuse the same block multiple times. This raises a question, can a model learn how to loop, per input, so it computes more efficiently. When a network has several distinct looped stages, is it better to give each stage its own independent halting controller (blockwise adaptive) or to keep a single shared halting controller for the whole network (global)? We evaluate four models sharing a common weight-tied backbone on a content addressed variable-hop retrieval task, comparing them at matched compute. From these results, the hypothesis is not supported. Per-block adaptive halting is beaten by the simpler global halting at matched compute and statistically identical from picking one good uniform depth. A positive however, is that the adaptive halting setup does perform real conditional computation.

CCS Concepts: • **Computing methodologies** → *Neural networks; Machine learning algorithms.*

Keywords: looped transformers, weight tying, adaptive computation, PonderNet, conditional computation, matched-compute evaluation, out-of-distribution generalization

1 MOTIVATION

A standard transformer applies the same sized stack of layers to every input, essentially spending the same compute on easy and hard problems. Looped, or weight-tied, transformers, instead reuse the same block multiple times. This raises a question, can a model learn how to loop, per input, so it computes more efficiently. More compute for hard steps, less compute for easier steps.

Research in this area partially exists with PonderNet-style adaptive halting. This solution proposes attaching a small attention head that determines a “halt-or-not” probability distribution, and let it learn this probability distribution over how many iterations to run. This project proposes

When a network has several distinct looped stages, is it better to give each stage its own independent halting controller (blockwise adaptive) or to keep a single shared halting controller for the whole network (global)?

2 THE TASK

To test this hypothesis, we propose using content addressed variable-hop retrieval. In our specific setup:

- Take a set of keys (16 in the sweep) and a random permutation that maps each key to a “next” key. This makes the pointer chain
- Give the model a start key and a target h , which defines the # of hops
- The model must follow the chain h times and output the key it lands on

Table 1: The four models compared. All share a common backbone of three weight-tied retrieval blocks and differ only in how the per-block loop count is determined.

Model	How loop counts are chosen	Role
Fixed	Hardcoded schedule (e.g., [2,2,2])	Control — establishes that depth matters and the task is learnable
Uniform	One learned count (applied identically to all three blocks)	Tests whether any learned depth helps
Global	One shared PonderNet controller (halts the whole network)	The standard/simpler adaptive baseline
Adaptive	Three independent controllers (one per block)	The contribution — per-block halting

h varies per example. An example needing 1 hop is genuinely easier than one needing 5, so a model that allocated compute by difficulty should have something to exploit. Training uses hops 1–5, evaluation additionally tests hops 6, 7, and 8 which are unseen during training. The held-out hops are the OOD generalization test, and aim to reveal whether the model learned the actual iterative algorithm of just memorized patterns up to $h = 5$.

2.1 Models compared

All 4 models used the same backbone, three weight-tied retrieval blocks, each of which can loop up to $r_{\max} = 8$ times. The only differ in how the loop count is detected (Table 1).

Table 2: Best-configuration accuracy for each model.

Model	Accuracy
Global	1.00
Adaptive & Uniform	~1.00
Fixed	0.995

Table 3: Comparison at matched compute.

Comparison (at matched compute)	Mean accuracy difference	Adaptive win rate
Adaptive vs. Global	-0.094 (adaptive worse)	6.5%
Adaptive vs. Uniform	+0.004 (tie)	34%

The mechanics of PonderNet were applied to the adaptive controllers exactly. At each step, a controller emits a “halt-now” probability, which are then combined into a probability distribution over stopping times, and the training loss is the halting-weighted average of the per-step-prediction losses. We used a KL term against a geometric prior to discourage the model from always running to the maximum loop count. This is the right soft-halting formula.

4 MAIN METHODOLOGY

The whole point of this experiment is to compare fair-compute values between models. Essentially, a model could always run more iterations to score higher, but the real question lies in whether this style of adaptive halting achieves more accuracy per unit of compute. For this context, we select the # of loops as the basic unit of compute, all models are compared at equal compute.

- For adaptive/uniform and fixed compute, it is the sum of the three per-block loop counts
- For global, each iteration runs all three blocks, so compute = shared set count $\times$ 3

5 FINDINGS

Every model solves the task at its best config (Table 2).

The comparison lies in the compute efficiency (Table 3).

From these results, the hypothesis is not supported. Per-block adaptive halting is beaten by the simpler global halting at matched compute and statistically identical from picking one good uniform depth.

A positive however, is that the adaptive halting setup does perform real conditional computation, reading 95% accuracy at ~ 6.4 expected blocks and 99% at ~ 7.8 . The machinery works, but doesn’t translate into a real advantage vs compared to a global controller.